

From Policy Text to Executable Obligations: A Grounded Verification Harness for Dependable In-Car LLM Agents

Radouane Boucekir

Damir Safin

Tomas Bueno Momcilovic fortiss GmbH, Munich, Germany {boucekir, safin, momcilovic}@fortiss.org

July 23, 2026

Abstract

Large language model (LLM) agents deployed in vehicles must satisfy a written operating policy on every turn: a single hallucinated identifier, omitted mandatory side-effect, or premature completion claim fails the task. We present *AgentGuardUtil*, our entry to CAR-bench Track 1, which treats the LLM as a fallible proposer inside a grounded verify-and-revise loop. Its core novelty is a *runtime policy compiler*: the natural-language policy shipped with each episode is compiled, once per policy, into typed machine-checkable rules, a subset of which receive an *executable* form. A deterministic obligation engine interprets these rules against live tool results and the simulated post-write state of the draft itself, emitting the exact remedial calls with computed arguments rather than natural-language reminders. Around this engine, ~ 25 deterministic gates (identifier provenance, schema and enum validity, gather-before-act, confirmation and future-time protocols) and an LLM critic produce tiered findings that drive a bounded revision loop tuned for the Pass^k metric. Nothing is benchmark-specific: all checks derive from the policy, tool schemas, and observations provided at run time.

1 Introduction

Context. LLM-based agents are moving rapidly from research prototypes into products that act on the physical world. In-vehicle voice assistants are an exemplary setting: the agent controls windows, climate, lighting, navigation, charging, and communication on behalf of a driver whose attention is limited, under a manufacturer policy that encodes safety and interaction rules. Here the cost of an unreliable action — a fabricated identifier, an omitted mandatory side-effect, a confidently wrong “done” — is not a benchmark point but a trust and safety incident.

The problem. Existing agent benchmarks largely evaluate under idealized conditions — complete information, available tools, unambiguous instructions — whereas deployed agents face incomplete or ambiguous requests, capabilities that may be missing, and domain policies that constrain behavior; CAR-bench [1] was built around exactly this gap, and

its baseline studies report that frontier models systematically prioritize task completion over admitting incapability. Concretely (Fig. 1), a simulated driver converses in closed loop with the agent, which observes and mutates a vehicle–world environment through 58 tools under a multi-page natural-language policy; tasks are scored on a *conjunction* of criteria (exact final *and* intermediate environment states, tool coverage, execution errors, automated and LLM-judged policy compliance, correct termination), and the headline metric over k repeated trials is

$$\text{Pass}^k = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \prod_{j=1}^k \mathbf{1}[\text{trial } j \text{ of } \tau \text{ passes}], \quad (1)$$

with $k=3$ on the hidden test set. Under an i.i.d. Bernoulli model with per-task success rate q_τ , $\mathbb{E}[\text{Pass}^k] = \frac{1}{|\mathcal{T}|} \sum_{\tau} q_\tau^k$: the metric is dominated by the *stable core* of tasks with $q_\tau \approx 1$, while near-coin-flip tasks contribute almost nothing ($0.5^3=0.125$). The challenge is therefore twofold: suppress the characteristic compliance failures (invented identifiers, skipped mandatory side-effects, non-canonical write sequences, refusals of computable requests, present-time answers to future-time constraints), and make correct behavior *repeatable* rather than merely likely.

Our approach. *AgentGuardUtil* interposes a verification layer between the LLM and the vehicle (Fig. 1): the LLM only *proposes*; a harness grounded in the episode’s own artifacts decides what may execute. At the first turn, the natural-language policy is compiled — once, and cached — into typed machine-checkable rules, part of which receive an *executable* form. On every draft action, a deterministic obligation engine evaluates these rules against live tool results and the simulated post-write state of the draft itself, computing the exact remedial calls the policy requires; ~ 25 further deterministic gates enforce identifier provenance, schema validity, and interaction protocols, and an LLM critic adjudicates the uncertain findings. Surviving findings drive a bounded revise-and-recheck loop before anything reaches the environment. Our contributions:

1. a **runtime policy compiler** producing typed rules with an executable sub-language, cached per policy hash (§2.2);

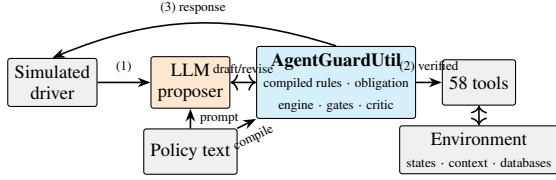


Figure 1: AgentGuardUtil in the CAR-bench loop [1]: the simulated driver (1) converses with the agent; the LLM only drafts, the harness verifies and revises before (2) tool execution against the vehicle–world environment and (3) the spoken response. The policy is both the LLM’s prompt and the harness’s compilation source.

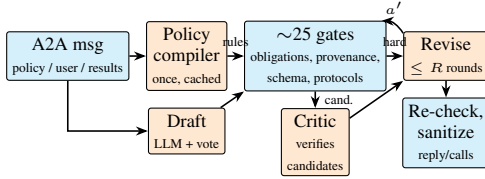


Figure 2: One agent turn. Orange components call an LLM; cyan components are deterministic. The compiler runs once per distinct policy; the gate suite and obligation engine run on every draft.

2. a **deterministic obligation engine** that evaluates executable rules on a projection of the environment state — including *simulation of the draft’s own writes* — and emits exact remedial tool calls (§2.3);
3. a **tiered verification loop** combining deterministic gates with an LLM critic, engineered for Pass^k stability (§2.4);
4. an **empirical variance analysis** on the public splits motivating these choices (§4).

The design composes chain-of-verification prompting [2], self-consistency [3], and tool-use agent patterns [4, 5], but moves the decisive checks out of the model and into code that the model cannot hallucinate around.

2 The AgentGuardUtil Harness

2.1 Turn Pipeline

Figure 2 and Algorithm 1 summarize one turn. The student LLM proposes a draft action a (a spoken reply and/or tool calls) given the history h and tool schemas $\mathcal{S}$. When the draft mutates state and voting is enabled, n samples are drawn and the modal *action set* (order-insensitive multiset of normalized calls) is kept — self-consistency applied only where errors are irreversible. The draft then enters verification (§2.4); surviving findings are injected as an internal note and the model revises, for at most R rounds ($R=2$). Because a revision may itself introduce a new defect, a final deterministic-only re-check runs once, optionally with a stronger *escalation* model. A text sanitizer strips non-speakable markup, and

Algorithm 1: VERIFIEDTURN($h, \mathcal{S}$)

```

1 if first turn for this policy  $\Pi$  then
2    $\mathcal{R} \leftarrow \text{COMPILE}(\Pi, \mathcal{S})$ ;           // cached by hash( $\Pi$ )
3  $L \leftarrow L \cup \text{EXTRACTTASKS}(\text{new user msg})$ ;
4  $a \leftarrow \text{MODALVOTE}(\{\text{LLM}(h, \mathcal{S})\}_{1..n})$ ;
5 for  $i \leftarrow 1$  to  $R$  do
6    $F \leftarrow F_{\text{hard}}(a, h, \mathcal{R}) \cup \text{CRITIC}(F_{\text{adv}}(a, h, \mathcal{R}))$ ;
7   if  $F = \emptyset$  then break;
8    $a \leftarrow \text{REVISE}(h, a, F)$ ;
9 if  $a$  was revised and  $F_{\text{hard}}(a) \neq \emptyset$  then
10   $a \leftarrow \text{REVISE}_{\text{esc}}(h, a, F_{\text{hard}}(a))$ ;           // safety net
11 return SANITIZE( $a$ )

```

A2A turn metrics (token usage, cost, latency across *all* internal calls) are attached to turn-final responses.

2.2 Runtime Policy Compilation

At the first turn of an episode the policy text Π (with volatile location/time fields normalized out) is compiled by an LLM into a rule set $\mathcal{R}$, cached process-wide under hash(Π) so compilation adds no per-task variance. Each rule $r = \langle \text{id}, \kappa, \mathcal{T}_r, \rho \rangle$ carries a type κ (*precondition*, *auto-action*, *confirmation*, *prohibition*, *disclosure*, *constraint*, *selection*), trigger tools $\mathcal{T}_r$ grounded against the live tool inventory, and an imperative requirement ρ . Compound clauses are split into atomic rules (one condition, one remedy). A second, focused pass translates state-conditional rules into an *executable* form

$$e_r = \langle g, \varphi, \triangleleft, v, \omega \rangle, \quad (2)$$

where g is a read tool, φ a field-name pattern with one wildcard, $\triangleleft \in \{<, \leq, >, \geq, =, \neq, \exists, \nexists\}$, v a threshold, and $\omega(x) = \langle u, \beta(x) \rangle$ an obligation template binding the matched item x into the arguments of tool u , validated against the declared enum of u ’s schema. The compiler sees real signatures $u(p_1, \dots, p_m)$; an obligation whose arguments cannot be validated is dropped, degrading that rule to advisory — the engine can only *add* precision, never force an invalid call.

2.3 Executable Obligations

Let σ_g be the latest parsed result of read g and $w_1, \dots, w_m$ the write calls executed after it, followed by the draft’s own writes W_a . The engine forms a *projected* state $\hat{\sigma} = \pi(\sigma_g; w_{1..m} \cdot W_a)$: a write matching some obligation template is *simulated* (the addressed fields are assigned the written value, with ALL-style enum members fanning out over all matching fields); a write that merely shares the field’s subject tokens *invalidates* only those fields; a write whose tool result reports an error invalidates rather than simulates. Evaluating conditions on the *post-write* projection is what lets a draft that itself creates a violation (e.g. opening all windows to 50% while the air conditioning is on) generate its own remediation. The computed obligation set is

$$\Omega(a, h) = \bigcup_{r \in \mathcal{R}_{\triangleright}} \{ \omega(x) : \varphi \sim x, \hat{\sigma}(x) \triangleleft v \} \setminus (E_h \cup A_a), \quad (3)$$

where $\mathcal{R}_\triangleright = \{r : \mathcal{T}_r \cap (W_h \cup W_a) \neq \emptyset\}$ are the triggered rules; obligations whose condition holds on $\hat{o}$ survive, minus calls already executed (E_h) or contained in the draft (A_a). When every matched item qualifies with identical residual arguments and the schema declares an ALL-style member, the set is collapsed to a single aggregate call — matching the canonical action granularity that intermediate-state scoring expects. Symmetric checks flag *excess*: writes on items the condition exempts, and aggregate calls where only a subset qualifies. Findings quote ready-to-execute calls with evidence (“window_driver_position= 25 > 20”), turning policy compliance from recall into transcription.

2.4 Tiered Verification and Variance Control

Around the obligation engine run ~ 25 deterministic gates, including: *identifier provenance* (every opaque id in outgoing arguments must appear in prior observations — a deterministic anti-hallucination oracle); *schema/enum validity* pre-execution; *gather-before-act* and preference-retrieval requirements; a *confirmation protocol* gate (confirmation-marked tools blocked until an intent question was affirmed); *completion* and *plan-completion* gates (no success claims without matching actions); a *refusal-read* gate (a can’t-do reply without ever reading the relevant subsystem must first perform the free read); and a *future-condition* gate (requests referencing arrival times or future windows must not be answered with current-moment filters, but by fetching unfiltered data and evaluating the constraint from returned fields).

Findings are *tiered*: hard findings (deterministically certain) always trigger revision; advisory candidates are passed to an LLM critic that verifies them against the transcript and tool schemas, and only confirmed items survive [2]. This tiering is a variance mechanism: under Eq. (1) a spurious revision on a correct draft is as costly as a missed defect, so uncertain signals must not force edits. Two further mechanisms serve Pass^k : revision is *bounded* with an oscillation valve (a write-set once revised away and re-proposed relaxes the gather gate instead of looping), and every layer is fail-safe (an internal exception contributes no findings rather than crashing the turn).

3 Generalization and Deployment

No gate, rule, tool name, or threshold is benchmark-specific: everything is derived at run time from the provided policy, tool schemas, and observations, so the harness transfers unchanged to the hidden set — and beyond it (unit tests include alien-domain policies such as smart-home and greenhouse control). The agent ships as a digest-pinned linux/amd64 image; every layer is switchable via `ENABLE_*` environment flags, and model, provider route, and sampling are env-configurable (LiteLLM). The submission configuration runs Claude Opus 4.8 via the direct Anthropic API and was validated end-to-end with the official evaluator image (GHCR option-C protocol). The code base carries 108 deterministic

Public split	Tasks	Pass ¹	Cost/task
Base	50	74.0%	\$0.054
Hallucination	48	62.5%	\$0.039
Disambiguation (3 runs)	31	48.4–58.1%	\$0.073–0.085

Table 1: Development results on the public training splits with a gemini-2.5-flash student in all harness roles (best base run; disambiguation shown as min–max over three full runs). The submitted configuration upgrades the student to Claude Opus 4.8.

unit tests and structured per-turn traces (drafts, fired gates, findings) for auditability.

4 Empirical Analysis

Table 1 reports development results with a deliberately weak and inexpensive student model, chosen to stress the harness rather than the LLM. The most consequential measurement is *run-to-run variance*: across repeated identical disambiguation runs, 42–44% of tasks flipped outcome at least once, and two same-configuration runs agreed on only 25/31 tasks. In the Bernoulli view of Eq. (1) this places a large task mass at intermediate q_τ , where the $k=3$ product annihilates score — e.g. the stable-pass core across three disambiguation runs was 9/31 tasks, against single-run Pass^1 of $\sim 52\%$. Failure forensics against ground-truth action chains attribute the residual gap to model-reasoning classes — guessed unstated argument values, arrival-time constraints answered with current-state reads, and non-canonical write granularity breaking intermediate-state subsets — each of which motivated a dedicated generic gate (§2.4); harness interventions themselves are verified regression-safe by construction (read-only remedies, conjunctive trigger conditions). These diagnostics, and the leaderboard dominance of stronger students on identical scaffolds, drove the submission choice of a frontier student model with the verification budget concentrated on deterministic, zero-variance checks.

5 Conclusion

AgentGuardUtil demonstrates that the reliability demands of in-vehicle assistants are best met not by prompting alone but by *compiling* the operating policy into executable obligations and letting deterministic code — grounded in live observations and simulated post-write state — decide what the model must do, verify, or refrain from. The design is benchmark-agnostic by construction and engineered for the repeatability that Pass^k rewards.

Code. <https://github.com/BoucheKirRedouane/car-bench-ijcai>

References

- [1] CAR-bench: Car Assistant Reliability Benchmark. IJCAI-ECAI 2026 Competition, Track 1. <https://car-bench.github.io/car-bench/>, 2026.
- [2] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston. Chain-of-verification reduces hallucination in large language models. *arXiv:2309.11495*, 2023.
- [3] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. In *ICLR*, 2023.
- [4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.
- [5] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv:2406.12045*, 2024.